

Ejercicio #3

Propiedades

#	Tipo Datos	Nombre	Visibilidad	Motivo	Clase
1	int	NoOrden	Private	Saber cuál es el número de orden.	Orden
2	String	Nombre	Private	El nombre de quien efectuó la orden.	Orden
3	String	Placa	Private	Saber la placa del vehículo.	Orden
4	String	Servicio	Private	Qué servicio necesita.	Orden
5	double	Costo	Private	Qué precio estimado va a tener.	Orden
6	List<Orden>	Ordenes	Private	Almacena todas la órdenes del día.	Controlador
7	Vista	vista	Private	Mostrar los datos solicitados	Controlador
8	Scanner	scanner	Public	Mostrar las opciones	Vista

Métodos

#	Tipo de Datos	Nombre	Visibilidad	Motivo	Clase
1	Void	main (args: String[])	Public	Inciar programa	Main
2	Void	Inciar	Public	Iniciar el combate	Controlador
3	Void	registrarOrden(Orden Servicio orden)	Public	Registra una orden.	Controlador
4	List <Ordenes >	consultarOrdenes()	Public	Mostrar las ordenes.	Controlador
5	Orden	buscar(int NoOrden)	Public	Buscar una orden por numero	Controlador
6	Void	modificar(int NoOrden, String nuevaDescripcion, double nuevoCosto)	Public	Modifica la descripción y el costo	Controlador
7	Void	cancelar(int NoOrden)	Public	Cancela una orden existente	Controlador
8	List <Orden<	BuscaRPorPlaca (String placa)	Public	Busca la orden según la placa.	Controlador
9	Double	CostoTotal()	Public	Calcula el total de las ordenes activas	Controlador
10	Double	Promedio()	Public	Calcula el promedio de los ordenes activas	Controlador

11	Orden	OrdenMayor()	Public	Encuentra la orden con mayor costo	Controlador
12	Int	CantidadOrden()	Public	Cuántas ordenes hay activas	Controlador
13	Boolean	ExistenciaOrden()	Public	Valida si el número de orden ya fue usado	Controlador
14	Void	validarOrden(Orden orden)	Public	Valida que toda la información este bien	Controlador
15	Void	mostrarMenu()	Public	Mostrar el menú de opciones	Vista
16	Void	mostrarOrden(Orden orden)	Public	Mostrar la orden	Vista
17	Void	mostrarOrdenes(List <Orden> ordenes)	Public	Muestra todas las ordenes	Vista
18	Void	mostrarCostos (double total, double promedio)	Public	Muestra el total y el promedio de costos	Vista
19	Void	mostrarCantida(int cantidad)	Public	Muestra la cantidad de ordenes	Vista
20	Void	Mensaje(String Mensaje)	Public	Muestra las confirmaciones o errores	Vista
21		Orden	Public	Crea una orden	Orden

		(int NoOrden, String nombre, String placa, String descripcion, double costo)			
22	Int	getNoOrden()	Public	Obtener el numero de orden	Orden
23	String	getNombre()	Public	Obtener el nombre	Orden
24	String	getPlaca()	Public	Obtener la placa	Orden
25	String	getDescripcion()	Public	Obtener la descripción	Orden
26	int	getCosto()	Public	Obtener el costo	Orden
27	void	setDescripcion()	Public	Modificar la descripcion	Orden
28	Void	setCosto()	Public	Modificar el costo	Orden
29	String	toString()	Public	Vuelve los datos a String	Orden

4)

La colección almacenará objetos de tipo Orden. Cada objeto contendrá el número de orden, nombre del propietario, placa del vehículo, descripción del servicio y costo estimado.

5)

List se utilizará para declarar el atributo de la colección dentro del Controlador mientras que ArrayList se utilizará para crear la colección en el constructor.

8)

Los valores iniciales de cada orden se proporcionarán mediante el constructor de OrdenServicio. Los datos serán solicitados al usuario antes de crear el objeto y luego se inicializa como un ArrayList.

9)

Cada orden se identificará mediante su número de orden, el cual deberá ser único. Para encontrarla, se recorrerá la colección y se comparará el número buscado con el número de cada objeto.

10)

Una orden se agregará utilizando el método add() y se eliminará con un remove().

11)

Se creará una lista vacía para almacenar las coincidencias. Después, se recorrerán todas las órdenes y se comparará la placa de cada una con la placa solicitada.

12)

Para obtener el valor total, se recorrerá la colección mediante un ciclo for-each y se acumulará el costo de cada orden y el promedio se calculará dividiendo el total entre la cantidad de órdenes.

13)

Se tomará inicialmente la primera orden como la de mayor costo. Luego, se recorrerá la colección y se comparará su costo con el de las demás órdenes.

14)

Las situaciones que pueden producir excepciones son:

- Introducir texto cuando se espera un número.
- Introducir un costo con un formato incorrecto.
- Registrar un número de orden repetido.
- Dejar vacío el nombre del propietario.
- Dejar vacía la placa o la descripción.
- Introducir un costo menor o igual que cero.
- Buscar una orden inexistente.
- Intentar modificar una orden inexistente.
- Intentar cancelar una orden inexistente.
- Solicitar la orden de mayor costo cuando no existen órdenes.
- Calcular el promedio cuando la colección está vacía.

15)

Se protegerán mediante try-catch las siguientes operaciones:

- Lectura del número de orden.
- Lectura del costo estimado.
- Registro de una orden con datos inválidos.
- Búsqueda de una orden.

- Modificación de una orden.
- Cancelación de una orden.
- Consulta de la orden de mayor costo cuando la colección esté vacía.

16)

El bloque finally mostrará un mensaje que indique que la operación terminó y que el usuario regresará al menú principal. Esta acción se ejecutará tanto si la operación fue exitosa como si se produjo una excepción. Su propósito es informar que el proceso terminó y demostrar que el sistema continuará funcionando después de controlar cualquier error.